

Лабораторная работа №5

Многофайловые проекты. Стандартная библиотека шаблонов.

СОДЕРЖАНИЕ

Задание №1.....	3
Задание №2.....	5
Задание №3.....	9
Задание №4.....	13

Цели и задачи работы: программирование и отладка программ формирования и обработки контейнеров, комбинации контейнеров.

Задание к работе: написать программу решения задачи в соответствии с индивидуальным вариантом.

Методика выполнения работы:

1. Разработать алгоритм решения задачи по индивидуальному заданию.
2. Написать и отладить программу решения задачи.
3. Протестировать работу программы на различных входных данных.

Замечание: программа из каждого задания должна быть реализована в виде многофайлового проекта. Необходимо вынести описание структур данных и объявление функций для работы в файл заголовка (*.h*), а реализацию функций разместить в отдельном *source*-файле. Основной файл *main.cpp* должен содержать только точку входа в программу и цикл обработки пользовательского ввода. Запрещается размещать логику обработки в основном файле.

Нумерация для пользователей с единицы. Использовать только контейнеры и структуры, все циклы должны быть Range-based for loop.

Требования к отчету:

Отчет должен содержать титульный лист, электронное содержание, перечень заданий, тексты программ и результаты работы программ в виде скриншотов.

Лабораторная содержит обязательный перечень понятий к защите. Преподаватель задает вопросы в процессе демонстрации студентом полученных навыков работы в интегрированной среде.

Перечень понятий к защите лабораторной работы 5:

Многофайловый проект: структура программы из нескольких файлов, модульность, разделение интерфейса и реализации. Файл заголовков: расширение .h, объявления функций, классов, структур. Файл исходного кода (source-файл): расширение .cpp, реализация функций. Защита от повторного включения: директивы #ifndef, #define, #endif, pragma once.

Понятие, история и характеристика STL, Ассоциативные контейнеры, Последовательные контейнеры. Контейнер vector: назначение, инициализация, принцип работы, Контейнер map: назначение, инициализация, принцип работы, Контейнер set: назначение, инициализация, принцип работы.

Определение функции, Типичный механизм возврата значений, Объявление функций (виды), Возврат нескольких значений из функций, Метод передачи параметров функции по значению: плюсы, минусы, технология, Метод передачи параметров функции по ссылке: плюсы, минусы, технология, Метод передачи параметров функции по константной ссылке: плюсы, минусы, технология, Метод передачи параметров функции по указателю: плюсы, минусы, технология, Куча, Принцип выбора метода передачи параметров функции, Возможности функций: перегрузка функций, Рекурсия (виды, примеры), Правила использования рекурсии, Шаблон функций.

Модель потока ввода-вывода, Иерархия потоков ввода-вывода, Перегрузка операторов, Перегрузка оператора ввода, вывода: особенности, примеры, Перегрузка арифметических операторов: особенности, примеры,

Директивы, Макросы, Препроцессор, Макро-константы, Макрофункции (многострочные), Predefined defines, Минусы использования макросов, Плюсы использования макросов, Правила использования макросов.

Материал для ознакомления:

<https://en.cppreference.com/cpp/container>

Задание №1¹

Учет товаров на складе

Реализовать программу для учета товаров на складе. Все ячейки на складе имеют свои адреса (например, A1739), адрес состоит из следующих обозначений:

- 1) A, Б, В – зона хранения (теплый, холодный склад или часть склада);
- 2) 17 – порядковый номер стеллажа;
- 3) 3 – порядковый номер вертикальной секции стеллажа;
- 4) 9 – порядковый номер полки.

В одну ячейку может быть помещен только один вид товара. В каждую ячейку помещается не более 10 единиц товара. Программа должна позволять добавлять товары в ячейки, просматривать состояние склада, убирать товар из ячейки. Конфигурация склада (количество зон, стеллажей, секций и полок) определяется согласно индивидуальному варианту из таблицы 1.

Для добавления товара в ячейку использовать команду *ADD* <наименование товара> <количество> <адрес ячейки>. *Пример: ADD Апельсины 8 A1739*. Так как размер ячейки ограничен 10 единицами товара, при попытке добавления большего количества товара пользователю должно выводиться соответствующее сообщение. Если ячейка занята другим товаром, также должно выводиться сообщение об ошибке.

Для удаления товара из ячейки использовать команду *REMOVE* <наименование товара> <количество> <адрес ячейки>. *Пример: REMOVE Апельсины 8 A1739*. Если в ячейке недостаточно товара для списания или товар не найден, пользователю должно выводиться соответствующее сообщение. При удалении всего количества товара из ячейки она считается свободной.

Для получения информации о состоянии склада использовать команду *INFO*. Команда должна выводить насколько процентов загружен склад, на сколько процентов загружена каждая зона склада, а также выводить содержание каждой ячейки, в которой есть хотя бы 1 единица товара, и выводить список адресов пустых ячеек. Процентные значения должны выводиться с точностью до двух знаков после запятой. Вывод информации должен быть структурирован по логическим блокам для удобства чтения.

¹Распределение вариантов по модулю 10

Таблица 1

Выбор конфигурации склада

Вариант	Кол-во зон хранения	Кол-во стеллажей в каждой зоне	Кол-во вертикальных секций стеллажа	Кол-во полок в вертикальной секции	Общая вместимость единиц товара
1	3	20	5	2	6000
2	2	15	3	5	4500
3	3	17	4	3	6120
4	1	12	7	4	3360
5	2	10	6	7	8400
6	3	16	8	6	23040
7	2	11	5	4	4400
8	3	14	4	8	13440
9	1	19	5	3	2850
10	2	13	7	9	16380

Пример работы программы

```

>>> ADD Апельсины 5 A1739
Добавлено 5 Апельсины в A1739
>>> ADD Яблоки 5 A1739
Ошибка: Ячейка A1739 занята товаром Апельсины
>>> ADD Апельсины 6 A1739
Ошибка: Превышена вместимость ячейки (максимум 10)
>>> REMOVE Апельсины 3 A1739
Удалено 3 Апельсины (остаток: 2)
>>> REMOVE Апельсины 4 A1739
Ошибка: Недостаточно товаров для удаления
>>> REMOVE Груши 1 A1739
Ошибка: Товар Груши не найден в ячейке A1739
>>> INFO
Загруженность склада: 0.03 %
Загруженность зоны А: 0.10 %
Заполненные ячейки:
A1739: Апельсины (2)
Пустые ячейки:
A1738, A1740, ... (список адресов)

```

Задание №2²

Вариант 1

Реализация электронной очереди

Реализовать программу для моделирования работы электронной очереди в поликлинике. На вход программе подается количество окон, способных обрабатывать посетителей. Каждое окно может обслуживать одного посетителя за раз.

Посетители добавляются в очередь командой *ENQUEUE*, которая принимает продолжительность посещения в минутах и выдает уникальный номер талона. Номера талонов формируются в формате *T* + три цифры с ведущими нулями (*T001*, *T002*, *T003* и так далее). Нумерация талонов начинается с единицы при каждом запуске программы. После того как введено нужное количество посетителей, вводится команда *DISTRIBUTE* для распределения очереди на все окна.

Распределение очереди посетителей на все окна должно быть выполнено таким образом, чтобы обработать очередь максимально быстро. Каждый следующий посетитель направляется в окно, где итоговое время обработки будет минимальным. При одинаковой итоговой загрузке нескольких окон выбор делается в пользу окна с меньшим порядковым номером. После распределения программа должна вывести для каждого окна общую сумму времени обслуживания и список талонов посетителей в порядке их назначения.

Пример 1

Введите количество окон:

```
>>> 3
>>> ENQUEUE 10
      T001
>>> ENQUEUE 10
      T002
>>> ENQUEUE 10
      T003
>>> ENQUEUE 5
      T004
>>> DISTRIBUTE
Окно 1 (15 минут): T001, T004
Окно 2 (10 минут): T002
Окно 3 (10 минут): T003
```

² Распределение вариантов по модулю 2

Пример 2

Введите количество окон:

```
>>> 2
>>> ENQUEUE 1
      T001
>>> ENQUEUE 1
      T002
>>> ENQUEUE 1
      T003
>>> ENQUEUE 10
      T004
>>> DISTRIBUTE
      Окно 1 (3 минут): T001, T002, T003
      Окно 2 (10 минут): T004
```

Вариант 2

Управление контейнерами в порту

Реализовать программу для моделирования работы порта по приему и отгрузке контейнеров. Контейнеры прибывают в порт последовательно и должны быть размещены на хранение в стеки. Каждый стек имеет ограничение по максимальной грузоподъемности в тоннах. Контейнеры размещаются в стеках по принципу *LIFO* (*последний пришел – первый ушел*). При размещении контейнер направляется в первый по порядку стек, где текущий вес не превысит лимит после добавления. Если ни один существующий стек не подходит, создается новый стек.

Для управления портом используются следующие команды. Команда *ARRIVE* <идентификатор> <вес> регистрирует прибытие контейнера с указанным идентификатором и весом в тоннах. Идентификатор состоит из буквы *C* и трех цифр с ведущими нулями (*C001*, *C002*, *C003* и так далее). Нумерация контейнеров начинается с единицы при каждом запуске программы.

Команда *LOAD* распределяет все накопленные контейнеры на секции судна. Контейнеры извлекаются из стеков в обратном порядке прибытия (*LIFO*) и направляются в секцию с наименьшей текущей суммой веса. При одинаковой загрузке нескольких секций выбор делается в пользу секции с меньшим порядковым номером. После распределения программа должна вывести для каждой секции общую сумму веса и список идентификаторов контейнеров в порядке их погрузки.

Пример 1

```
Введите максимальный размер стека:
>>> 20
Введите количество секций судна:
>>> 2
>>> ARRIVE C001 5
    Контейнер C001 размещен в стек 1
>>> ARRIVE C002 8
    Контейнер C002 размещен в стек 1
>>> ARRIVE C003 7
    Контейнер C003 размещен в стек 1
>>> ARRIVE C004 10
    Контейнер C004 размещен в стек 2
>>> LOAD
    Секция 1 (15 тонн): C004, C001
    Секция 2 (15 тонн): C003, C002
```

Пример 2

```
Введите максимальный размер стека:
>>> 15
Введите количество секций судна:
>>> 3
>>> ARRIVE C001 3
Контейнер C001 размещен в стек 1
>>> ARRIVE C002 3
Контейнер C002 размещен в стек 1
>>> ARRIVE C003 3
Контейнер C003 размещен в стек 1
>>> ARRIVE C004 10
Контейнер C004 размещен в стек 2
>>> LOAD
Секция 1 (10 тонн): C004
Секция 2 (6 тонн): C003, C001
Секция 3 (3 тонн): C002
```


Задание №3³

Многофайловый проект ENUM C++

Необходимо реализовать систему хранения и обработки информации по заданию в соответствии с вариантом из таблицы 2:

Таблица 2

Выбор систем хранения и обработки информации

Вариант	Задание
1	График движения поездов на железной дороге
2	График движения трамваев
3	График движения самолетов
4	График движения троллейбусов
5	График движения поездов в метро

Реализовать многофайловый проект, предусматривающий обработку следующих запросов:

Вариант	Задание	Расшифровка
1	CREATE_TRAIN <i>train</i> N <i>town</i> ₁ <i>town</i> ₂ ... <i>town</i> _N	Создание поезда с именем <i>train</i> , который проходит через города <i>town</i> ₁ , <i>town</i> ₂ , ..., <i>town</i> _N
	TRAINS_FOR_TOWN <i>town</i>	Вывод всех поездов, которые проходят через город <i>town</i>
	TOWNS_FOR_TRAIN <i>train</i>	Вывод всех городов, которые проезжает поезд с именем <i>train</i> . Для каждого города прописать, какие поезда проезжают этот город (не включая <i>train</i>)
	TRAINS	Отобразить все поезда с указанием городов-остановок
2	CREATE_TRAM <i>tram</i> N <i>stop</i> ₁ <i>stop</i> ₂ ... <i>stop</i> _N	Создание трамвая с именем <i>tram</i> , который проходит через остановки <i>stop</i> ₁ , <i>stop</i> ₂ , ..., <i>stop</i> _N
	TRAMS_IN_STOP <i>stop</i>	Вывод всех трамваев, которые проходят через остановку <i>stop</i>

³Распределение вариантов по модулю 5

Вариант	Задание	Расшифровка
	STOPS_IN_TRAM <i>tram</i>	Вывод всех остановок, которые проезжает трамвай с именем <i>tram</i> . Для каждой остановки прописать, какие трамваи идут через эту остановку (не включая <i>tram</i>)
	TRAMS	Отобразить все трамваи с указанием остановок
3	CREATE_PLANE <i>plane</i> N <i>town₁</i> <i>town₂</i> ... <i>town_N</i>	Создание самолета с именем <i>plane</i> , который пролетает через города <i>town₁</i> , <i>town₂</i> , ..., <i>town_N</i>
	PLANES_FOR_TOWN <i>town</i>	Вывод всех самолетов, которые пролетают через город <i>town</i>
	TOWNS_FOR_PLANE <i>plane</i>	Вывод всех городов, которые пролетает самолет с именем <i>plane</i> . Для каждого города прописать, какие самолеты пролетают этот город (не включая <i>plane</i>)
	PLANES	Отобразить все самолеты с указанием городов, которые они пролетают
4	CREATE_TRL <i>trl</i> N <i>stop₁</i> <i>stop₂</i> ... <i>stop_N</i>	Создание троллейбуса с именем <i>trl</i> , который проходит через остановки <i>stop₁</i> , <i>stop₂</i> , ..., <i>stop_N</i>
	TRLS_IN_STOP <i>stop</i>	Вывод всех трамваев, которые проходят через остановку <i>stop</i>
	STOPS_IN_TRL <i>trl</i>	Вывод всех остановок, которые проезжает троллейбус с именем <i>trl</i> . Для каждой остановки прописать, какие троллейбусы идут через эту остановку (не включая <i>trl</i>)
	TRLS	Отобразить все троллейбусы с указанием остановок
5	CREATE_METRO <i>line</i> N <i>station₁</i> <i>station₂</i> ... <i>station_N</i>	Создание линии метро с именем <i>line</i> , которая проходит через станции

Вариант	Задание	Расшифровка
		$station_1, station_2, \dots, station_N$
	METROS_IN_STOP <i>station</i>	Вывод всех линий метро, которые проходят через станцию <i>station</i>
	STOPS_IN_METRO <i>line</i>	Вывод всех станций, которые проходит линия с именем <i>line</i> . Для каждой станции прописать, какие линии метро идут через эту станцию (не включая <i>line</i>)
	METROS	Отобразить все линии метро с указанием станций

Все команды хранить в классе enum, например:

```
enum class Type {
    CREATE_TRAIN,
    TRAINS_FOR_TOWN,
    TOWNS_FOR_TRAIN,
    TRAINS
};
```

Пример работы программы (трамваи)

```
>>> TRAMS
Ошибка: Трамваи не найдены
>>> TRAMS_IN_STOP Marksa
Ошибка: Остановка Marksa не найдена
>>> STOPS_IN_TRAM 18
Ошибка: Трамвай 18 не найден
>>> CREATE_TRAM 18 4 Student Marksa TVset Cosmos
Трамвай 18 создан
>>> CREATE_TRAM 15 4 Student Cosmos TVset Titova
Трамвай 15 создан
>>> CREATE_TRAM 15 3 Student NGTU NSU
Ошибка: Трамвай с именем 15 уже создан
>>> CREATE_TRAM 16 1 Student
Ошибка: Трамвай не может быть создан с одной остановкой
>>> CREATE_TRAM 16 2 NSU NSU
Ошибка: Трамвай не может быть создан с двумя одинаковыми
остановками
>>> TRAMS_IN_STOP Student
Трамваи на остановке Student: 15, 18
>>> STOPS_IN_TRAM 15
Остановки трамвая 15: Student Cosmos TVset Titova
```

```
Остановка Student: 18
Остановка Cosmos: 18
Остановка TVset: 18
Остановка Titova: -
>>> TRAMS
Трамвай 15: Student Cosmos TVset Titova
Трамвай 18: Student Marksa TVset Cosmos
```

Задание №4⁴
Комбинация контейнеров

Вариант 1

Реализовать автоматизированную систему учета дружеских связей между людьми в виде многофайлового проекта:

Запрос	Расшифровка	Комментарий
FRIENDS <i>i j</i>	Записывает <i>i</i> , <i>j</i> как друзей	-
COUNT <i>i</i>	Подсчет количества друзей <i>i</i>	Результат – число
QUESTION <i>i j</i>	Проверка, дружат ли <i>i</i> и <i>j</i>	Результат – No/Yes

Придумать минимум 5 тестов с разными значениями (в том числе, повторяющимися).

Пример работы программы

Введите количество запросов (N):

```
>>> 9
>>> FRIENDS Peter Goward
    Peter и Goward теперь друзья
>>> FRIENDS Goward Peter
    Goward и Peter уже друзья
>>> FRIENDS Peter Peter
>>> FRIENDS Goward Sally
    Goward и Sally теперь друзья
>>> COUNT Goward
    2
>>> COUNT Justin
    0
>>> QUESTION Goward Peter
    Yes
>>> QUESTION Peter Peter
    Yes
>>> QUESTION Justin Jenny
    No
```

⁴ Распределение вариантов по модулю 5

Вариант 2

Реализовать программу для ведения справочника регионов России с административными центрами в виде многофайлового проекта:

Запрос	Расшифровка	Комментарий
CHANGE <i>region center</i>	Создание региона <i>region</i> с административным центром <i>center</i>	Новый регион <i>region</i> с административным центром <i>center</i>
RENAME <i>old_center new_center</i>	Переименование административного центра со старым названием <i>old_center</i> в административный центр с новым названием <i>new_center</i>	Административный центр <i>old_center</i> переименован в административный центр <i>new_center</i>
ABOUT <i>region</i>	Вывод административного центра <i>center</i> введенного региона <i>region</i>	Регион <i>region</i> имеет административный центр <i>center</i>
ALL	Вывод всех регионов и их административных центров	<i>region₁ - center₁, region₂ - center₂, ..., region_n - center_n</i>

Придумать минимум 5 тестов с разными значениями (в том числе, повторяющимися).

Пример

Введите количество запросов (N):

```
>>> 11
>>> CHANGE Sibir Novo-nikolaevsk
Новый регион Sibir с административным центром Novo-nikolaevsk
>>> CHANGE Moscow Novo-nikolaevsk
Ошибка: Novo-nikolaevsk уже является административным центром
другого региона
>>> CHANGE Sibir NovoSibirsk
Ошибка: Регион Sibir уже создан
>>> CHANGE Moscow Zelenograd
Новый регион Moscow с административным центром Zelenograd
>>> RENAME Zelenograd MoscowFO
Административный центр Zelenograd переименован в
административный центр MoscowFO
>>> RENAME MoscowFO MoscowFO
Ошибка: Нельзя переименовать MoscowFO в то же название
>>> RENAME KazanFO PermFO
Ошибка: Административный центр KazanFO не найден
>>> RENAME Novo-Nikolaevsk MoscowFO
Ошибка: MoscowFO уже является административным центром другого
региона
>>> ABOUT Moscow
Регион Moscow имеет административный центр MoscowFO
>>> ABOUT Kazan
Ошибка: Регион Kazan не найден
>>> ALL
Sibir - Novo-nikolaevsk, Moscow - MoscowFO
```

Вариант 3

Реализовать программу для автоматизации работы с расписанием занятий студентов в виде многофайлового проекта:

Запрос	Расшифровка	Комментарий
CLASS i s	Установить дисциплину s в день i текущего месяца	Добавлена дисциплина s на день i
NEXT	Смена месяца на следующий	Переход на следующий месяц
VIEW i	Организовать вывод всех дисциплин дня i	В день i занятия в университете: s
	При отсутствии дисциплин выдать соответствующее сообщение	В день i мы свободны!

При переходе на новый месяц пары сохраняются строго по дням и могут повторяться, отсчет ведется с января. Если день не существует в новом месяце (например, 30 число в феврале), пары переносятся на последний день текущего месяца. Количество дней в каждом месяце:

28 дней	30 дней	31 день
Февраль	Январь, Апрель, Июнь, Август, Октябрь, Декабрь	Март, Май, Июль, Сентябрь, Ноябрь

Придумать минимум 5 тестов с разными значениями (в том числе, повторяющимися).

Пример работы программы

Введите количество запросов (N):

```
>>> 10
>>> CLASS 5 INFORMATICA
Добавлена дисциплина INFORMATICA на день 5
>>> CLASS 5 INFORMATICA
Ошибка: Дисциплина INFORMATICA уже есть в этот день
>>> CLASS 30 YAP
Добавлена дисциплина YAP на день 30
>>> NEXT
Переход на следующий месяц
>>> VIEW 5
В день 5 занятия в университете: INFORMATICA
>>> VIEW 30
Ошибка: В этом месяце всего 28 дней
>>> VIEW 28
В день 28 занятия в университете: YAP
>>> VIEW 1
```

```
        В день 1 мы свободны!  
>>> NEXT  
        Переход на следующий месяц  
>>> VIEW 31  
        В день 31 мы свободны!
```


Вариант 4

Реализовать программу для автоматизации работы с потоком студентов факультета в виде многофайлового проекта:

Запрос	Расшифровка	Комментарий
NEW_STUDENTS <i>number</i>	>0: Добавить в общий список <i>number</i> студентов	Добавлено <i>number</i> студентов
	<0: Удалить из списка на отчисление <i>number</i> студентов	Удалено <i>number</i> студентов
SUSPICIOUS <i>number_student</i>	Студент с порядковым номером <i>number_student</i> помечается как кандидат на отчисление и попадает в списки на отчисление	Студент <i>number_student</i> стал кандидатом на отчисление
IMMORTAL <i>number_student</i>	Студент с порядковым номером <i>number_student</i> помечается как неприкасаемый, и он уходит из списков на отчисление	Студент <i>number_student</i> стал неприкасаемым
TOP-LIST	Вывод отсортированного списка студентов, входящих в списки на отчисление	Список студентов на отчисление: <i>Student₁</i> , <i>Student₂</i> , ..., <i>Student_N</i>

Придумать минимум 5 тестов с разными значениями (в том числе, повторяющимися).

Пример работы программы

Введите количество запросов (N):

```

>>> 11
>>> NEW_STUDENTS 20
Добавлено 20 студентов
>>> NEW_STUDENTS -5
Ошибка: Нет кандидатов на отчисление
>>> SUSPICIOUS 10
Студент 10 стал кандидатом на отчисление
>>> SUSPICIOUS 10
Ошибка: Студент 10 уже является кандидатом на отчисление
>>> NEW_STUDENTS -2
Ошибка: невозможно отчислить больше студентов, чем есть их в
списках на отчисление

```

```
>>> IMMORTAL 10
Студент 10 стал неприкасаемым
>>> NEW_STUDENTS -1
Ошибка: Нет кандидатов на отчисление
>>> IMMORTAL 11
Ошибка: Студент 11 не может стать неприкасаемым, поскольку он
не находится в списках на отчисление
>>> SUSPICIOUS 11
Студент 11 стал кандидатом на отчисление
>>> SUSPICIOUS 12
Студент 12 стал кандидатом на отчисление
>>> TOP-LIST
Список студентов на отчисление: Студент 11, Студент 12
```

Вариант 5

Реализовать программу для автоматизации работы городской библиотеки в виде многофайлового проекта:

Запрос	Расшифровка	Комментарий
ADD_BOOK <i>book_id name</i>	Регистрация новой книги с идентификатором <i>book_id</i> и названием <i>name</i>	Книга <i>book_id name</i> добавлена
BORROW <i>book_id reader_id</i>	Выдача книги с идентификатором <i>book_id</i> читателю с идентификатором <i>reader_id</i>	Книга <i>book_id</i> выдана читателю <i>reader_id</i>
RETURN <i>book_id</i>	Возврат книги с идентификатором <i>book_id</i> обратно в библиотеку	Книга <i>book_id</i> возвращена в библиотеку
BOOK_STATUS <i>book_id</i>	Вывод текущего статуса книги с идентификатором <i>book_id</i> (в библиотеке/у читателя)	Книга <i>book_id</i> находится в библиотеке
		Книга <i>book_id</i> находится у читателя <i>reader_id</i>
READER_BOOKS <i>reader_id</i>	Вывод списка всех книг, которые сейчас находятся у читателя с идентификатором <i>reader_id</i>	Читатель <i>reader_id</i> имеет книги: <i>book_id₁</i> , <i>book_id₂</i> .

Каждая книга имеет уникальный идентификатор и может находиться либо в библиотеке, либо у читателя. Каждый читатель может брать не более 2 книг.

Придумать минимум 5 тестов с разными значениями (в том числе, повторяющимися).

Пример работы программы

Введите количество запросов (N):

```
>>> 14
>>> ADD_BOOK B001 WarAndPeace
      Книга B001 WarAndPeace добавлена
>>> ADD_BOOK B001 AnnaKarenina
      Ошибка: Книга с идентификатором B001 уже существует
>>> ADD_BOOK B002 AnnaKarenina
      Книга B002 AnnaKarenina добавлена
```

```
>>> ADD_BOOK B003 Idiot
      Книга B003 Idiot добавлена
>>> BORROW B001 R001
      Книга B001 выдана читателю R001
>>> BORROW B004 R001
      Ошибка: Книги с идентификатором B004 не существует
>>> BORROW B002 R001
      Книга B002 выдана читателю R001
>>> BORROW B003 R001
      Ошибка: Книгу нельзя выдать читателю R001, поскольку у него
превышен лимит
>>> RETURN B004
      Ошибка: Книги с идентификатором B004 не существует
>>> RETURN B002
      Книга B002 возвращена в библиотеку
>>> BOOK_STATUS B002
      Книга B002 находится в библиотеке
>>> BOOK_STATUS B001
      Книга B001 находится у читателя R001
>>> READER_BOOKS R001
      Читатель R001 имеет книги: B001
>>> READER_BOOKS R999
      Читатель R999 имеет книги: -
```